

Augura

Un seul déploiement. Sept modules autonomes.

L'architecture du backend de production d'Augura : un monolithe modulaire Python/FastAPI déployé sur Modal, Supabase en porte de données unique, et un contrat de types qui court du SQL jusqu'au front.

La valeur est prouvée. La robustesse se construit.

Le backend actuel est une fracture non assumée : onze fonctions serverless Vercel en JavaScript portent les agents LLM et les proxies, pendant que le moteur statistique — bootstrap $N = 1\,000$, estimateurs ATE, ATT, LME, IPW — vit dans des scripts Python détachés de l'API. Deux runtimes, deux systèmes de types, aucun pont.

Les limites sont atteintes : l'agent de profiling plafonne au timeout de 300 secondes, aucun mécanisme de jobs longs ni de cron, RLS désactivé sur les tables, l'état du workflow confié au sessionStorage du navigateur, et le mode démo entremêlé au code de production. **Chaque nouvelle capacité coûte plus cher que la précédente.**

Deux runtimes par accident. Un seul par décision.

DÉCISION

Monolithe modulaire Python/FastAPI, déployé sur Modal. Supabase — instance neuve — comme base, auth et stockage.

PÉRIMÈTRE

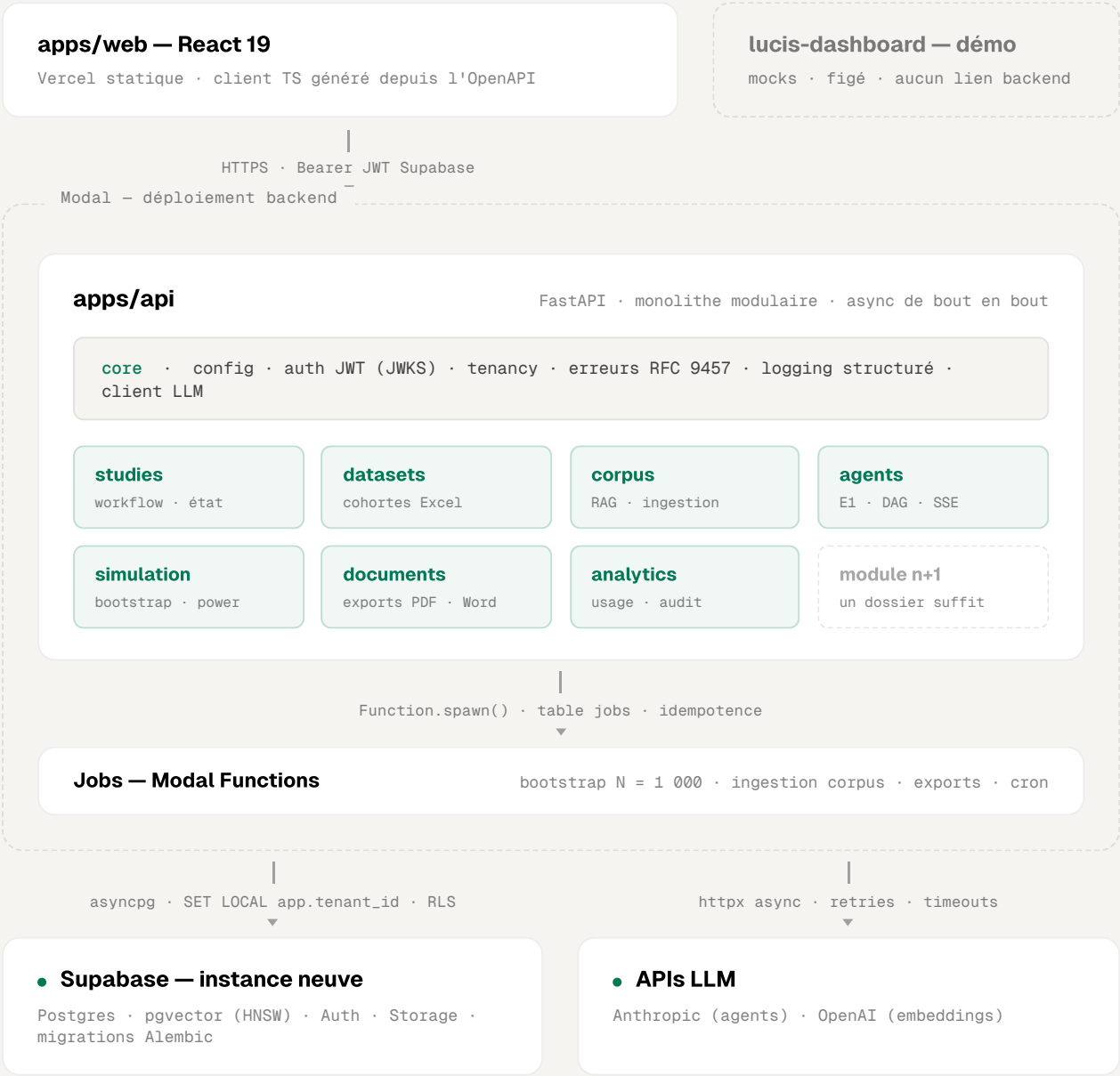
Remplace intégralement les fonctions Vercel et les scripts offline. La démo actuelle devient un artefact frontend isolé.

MÉTHODE

Big bang outillé : tests de caractérisation issus des fixtures réelles ; bascule uniquement quand la parité est prouvée.

Une seule porte

Le front parle à une seule porte : l'API. L'API parle à une seule base : Supabase. Tout calcul lourd sort du chemin des requêtes et devient une fonction Modal.



Chaque module expose `router` · `service` · `schemas` · `repo` et n'importe jamais l'intérieur d'un autre — la frontière est vérifiée par `import-linter` en CI. Un module qui grossit s'extrait en service sans réécriture : son contrat public existe déjà.

Pourquoi Python

Pas une préférence : un constat structurel. Le cœur scientifique d'Augura est en Python et n'en sortira pas. Tout autre choix de langage backend impose deux runtimes à vie.

Le cœur métier est intransportable

ATE, ATT, LME, IPW, bootstrap, power analysis : scipy, statsmodels, sklearn. Aucun équivalent crédible en JavaScript ; golang (Go) est embryonnaire. Python unifie l'API, les agents et la science.

L'IA y est first-class

SDK Anthropic natif, sorties d'agents validées par Pydantic avec retry de réparation, LangSmith, pipelines d'embeddings. L'écosystème où tout arrive en premier.

Un contrat de types de bout en bout

Pydantic v2 → OpenAPI → client TypeScript généré. La validation runtime, la documentation et le typage front sortent du même geste. Le front n'écrit jamais un appel à la main.

Modal est Python-natif

L'API entière se déploie en une décoration ASGI. Les jobs longs deviennent des fonctions — sans Celery, sans Redis, sans queue à opérer. Cron intégré, GPU accessible.

La « lenteur de Python » est un débat pour un profil de charge qu'Augura n'a pas.

Le backend est I/O-bound : la latence vit dans les appels LLM — des secondes — et dans la base, pas dans l'interpréteur. Le calcul lourd est vectorisé numpy, du C sous le capot. Go ne gagnerait que sur du débit HTTP massif à calcul léger ; l'usage B2B d'Augura en est loin.

CE QUI COMPTE ICI

- Un seul cerveau backend : API, agents et statistiques dans le même runtime.
- Des sorties d'agents validées par schéma, pas parsées à la main.
- Des jobs longs sans infrastructure de queue à opérer.

CE QUE L'ON CONCÈDE, EN CONNAISSANCE

- Node : types partagés sans génération, cold starts plus courts.
- Go : binaire statique, empreinte mémoire minime.
- Aucun des deux ne compense le premier point.

Sept modules, une règle

Chaque module est une tranche verticale autonome : `router` · `service` · `schemas` · `repo`. Un module n'importe jamais l'intérieur d'un autre — uniquement le socle et les interfaces publiques. La règle n'est pas une promesse : la CI casse si elle est violée.



core le socle — pas un module

Config fail-fast, vérification JWT (JWKS Supabase), résolution du tenant, hiérarchie d'erreurs, logging structuré, client LLM partagé, événements de domaine. Zéro logique métier ; aucun import vers les modules.



studies

Cycle de vie des études, état du workflow persisté et versionné en base — fini le sessionStorage —, membres et rôles par étude. Débloque multi-appareils et collaboration.



datasets

Upload des cohortes vers Supabase Storage, parsing et validation pandas hors event loop, profil de colonnes, mapping des variables.



corpus

Ingestion self-service — PDF, extraction, chunking, embeddings — vers pgvector ; retrieval en SQL direct ; corpus global partagé et corpus privé par tenant.



agents

E1 profiling en SSE multi-tour, DAG causal, gap-detection, variable-check — sur un runtime commun : boucle tool-use, sorties Pydantic, même protocole NDJSON que le front connaît.



simulation

Bootstrap à la demande — job Modal sur image scientifique dédiée — et approximation analytique synchrone, calibrée par outcome. Le précalculé manuel disparaît.



documents

Génération de protocoles et rapports : état complet de l'étude, LLM et template, rendu PDF ou Word, URL signée Supabase Storage. Asynchrone, en job.



analytics

Usage authentifié — le soft-auth disparaît —, audit trail alimenté par l'outbox d'événements, statistiques d'administration, coûts et latences LLM enregistrés par run d'agent.

Trois chemins, une discipline

Tout ce que fait le backend passe par l'un de ces trois chemins. Chacun a son budget de latence, son mode d'échec et son contrat.

Requête standard

p95 < 300 ms hors LLM

01 client TS généré — 02 JWT vérifié (JWKS) — 03 tenant résolu — 04 service — 05 repo + RLS
— 06 réponse Pydantic

La session ouvre chaque transaction par SET LOCAL app.tenant_id ; les policies RLS s'y adossent. Le scoping est appliqué deux fois — dans les repos, et par Postgres lui-même. L'oubli d'un filtre ne peut pas fuiter des données entre tenants.

Agent E1 — streaming

premier token < 2 s · sans plafond 300 s

01 POST /agents/profiling/stream — 02 boucle tool-use — 03 retrieval **corpus** —
04 events NDJSON streamés — 05 état d'étude persisté

Heartbeats pendant l'attente ; le flux se termine toujours par un event done ou error — jamais de spinner infini. Sorties structurées validées par Pydantic, avec un retry de réparation avant échec franc.

Job lourd — simulation, ingestion, export

202 immédiat · suivi par jobs

01 POST /simulations — 02 insert **jobs** + Function.spawn() — 03 202 · job_id —
04 worker met à jour la progression — 05 résultats lus par l'API

Clé d'idempotence obligatoire : un double-clic ne lance pas deux bootstraps. Échec = statut failed, erreur détaillée, alerte Sentry, relance manuelle. L'image scientifique — numpy, scipy, statsmodels — reste séparée de l'image API.

Le mode démo disparaît du backend.
La production n'a qu'un seul chemin de code.

Robuste par construction

La robustesse n'est pas une revue de code : c'est un ensemble de règles que la machine fait respecter. Trois familles, toutes vérifiées automatiquement.

TYPAGE

- **pyright strict** en CI — pas d'exception.
- Pydantic v2 à **toutes les frontières** : requêtes, réponses, settings, jobs, sorties d'agents.
- NewType pour TenantId, StudyId, JobId.
- Any et type: ignore interdits sans justification écrite.
- Les modèles SQLAlchemy ne sortent jamais d'un module.

EXÉCUTION

- Async de bout en bout ; l'event loop ne fait **que de l'I/O**.
- Tout calcul au-delà de ~100 ms sort de l'API : thread ou job Modal.
- Cache des réponses d'agents déterministes — mêmes entrées, réponse servie.
- Pagination keyset ; index pgvector HNSW.
- Budgets tenus : p95 < 300 ms hors LLM, premier token SSE < 2 s.

ERREURS

- Hiérarchie AppError ; réponses uniformes **RFC 9457**.
- LLM indisponible = **503 explicite**, jamais d'attente infinie.
- Sortie structurée invalide : un retry de réparation, puis échec franc.
- SSE : heartbeats, fin systématique par done ou error.
- Jobs échoués : erreur détaillée, alerte Sentry, relance manuelle.

LA CI CASSE SI

- un module importe l'intérieur d'un autre — contrat import-linter ;
- le client TypeScript a dérivé du contrat OpenAPI — drift check ;
- pyright strict, ruff ou un test échoue — aucun contournement ;
- un test d'isolation RLS prouve qu'un tenant lit l'autre.

Ces règles existent pour une raison simple : une équipe courte, augmentée d'agents, ne relit pas tout. **Ce que la machine vérifie n'a pas besoin d'être surveillé.**

Modal en production, bascule prouvée

Runtime

L'API en `@modal.asgi_app()` sur image légère uv — `min_containers = 1` en prod, zéro cold start utilisateur. Jobs sur image scientifique séparée ; cron intégré pour le récurrent.

Environnements

Modal dev et prod : secrets distincts, instances Supabase distinctes, migrations Alembic versionnées. Le front reste sur Vercel et pointe le domaine API.

Chaîne CI

`ruff · pyright strict · import-linter · pytest · drift` check du client TS — puis `modal deploy` sur main. Rien ne part en prod sans que tout soit vert.

Observabilité

structlog JSON avec `request_id`, Sentry pour les erreurs, LangSmith pour les traces d'agents, coûts et latences LLM par run en base — visibles dans analytics.

01	Fondations	scaffold du monorepo, verrous qualité, CI, healthcheck déployé sur Modal dev
02	Socle + studies	auth JWT, tenancy, premier module, client TS généré — le front migré boote
03	datasets + corpus	upload de cohortes, ingestion ; transfert du corpus existant sans ré-embedding
04	agents	caractérisation d'abord ; E1 SSE porté en dernier des quatre
05	simulation + jobs	infrastructure de jobs, bootstrap à la demande, calibration par outcome
06	documents + analytics	génération PDF et Word, usage authentifié, audit
07	Bascule	parité prouvée, front de production basculé, démo figée en artefact isolé

La bascule n'est pas une date.
C'est un critère.

Golden tests verts — les fixtures réelles de la démo rejouées contre le nouveau backend —, e2e verts, isolation RLS prouvée. Tant que ce n'est pas vert, l'ancien backend reste en service.

Augura

**Le prototype a prouvé la valeur.
L'architecture prouvera la durée.**

Spécification : `docs/specs/2026-06-11-augura-backend-architecture-design.md`

Plan d'exécution : `docs/plans/2026-06-11-phase-1-fondations.md`